

Manual Técnico - EffiCode Analyzer

Sistema de Análisis de Complejidad Algorítmica

Versión: 1.0.0

Fecha: Diciembre 2025

Curso: Análisis y Diseño de Algoritmos

Universidad: Universidad de Caldas

Tabla de Contenidos

- [Descripción General](#)
- [Requisitos del Sistema](#)
- [Arquitectura del Sistema](#)
- [Estructura del Código](#)
- [Dependencias](#)
- [Instalación](#)
- [Configuración](#)
- [Ejecución](#)
- [API REST - Endpoints](#)
- [Solución de Problemas](#)
- [Apéndice](#)

1. Descripción General

EffiCode Analyzer es una aplicación web full-stack diseñada para realizar el análisis estático y simbólico de la complejidad algorítmica de pseudocódigo estilo Cormen (Θ , O , Ω). El sistema emplea una arquitectura híbrida que combina motores matemáticos determinísticos con inteligencia artificial para la validación y enriquecimiento de las explicaciones.

Características Clave

Característica	Propósito	Tecnología Principal
----------------	-----------	----------------------

Análisis Estático	Cálculo de sumatorias y manipulación de funciones de costo.	AST, SymPy
Recurrencias	Resolución formal de ecuaciones de recurrencia.	RecurrenceSolver (7 métodos)
Validación IA	Verificación semántica de la complejidad deducida y traducción de lenguaje natural.	Google Gemini 2.5-pro
Clasificación Neural	Predicción de complejidad por patrones estructurales.	MLP implementado en NumPy
Visualización	Muestra el Árbol de Sintaxis Abstracta (AST) y el Árbol de Recursión.	Graphviz, KaTeX

2. Requisitos del Sistema

2.1 Software Requerido

Herramienta	Versión Mínima	Propósito
Python	3.10+	Backend API y Lógica de Análisis
Node.js	18.0+	Frontend build (React/Vite)
npm	9.0+	Gestión de paquetes JS

Git	2.30+	Control de versiones
Graphviz	2.40+	Generación de AST visual

2.2 Hardware Mínimo Recomendado

Componente	Mínimo	Recomendado
Procesador	2 núcleos, 2.0 GHz	4 núcleos, 2.5 GHz
RAM	4 GB	8 GB
Disco	500 MB libres	1 GB libres
Conexión	Internet estable	Banda ancha (para LLM)

2.3 Verificar Instalaciones

Es crucial verificar que las dependencias base del sistema estén disponibles antes de la instalación:

Bash

```
# Verificar Python
python3 --version
```

```
# Verificar Node.js
node --version
```

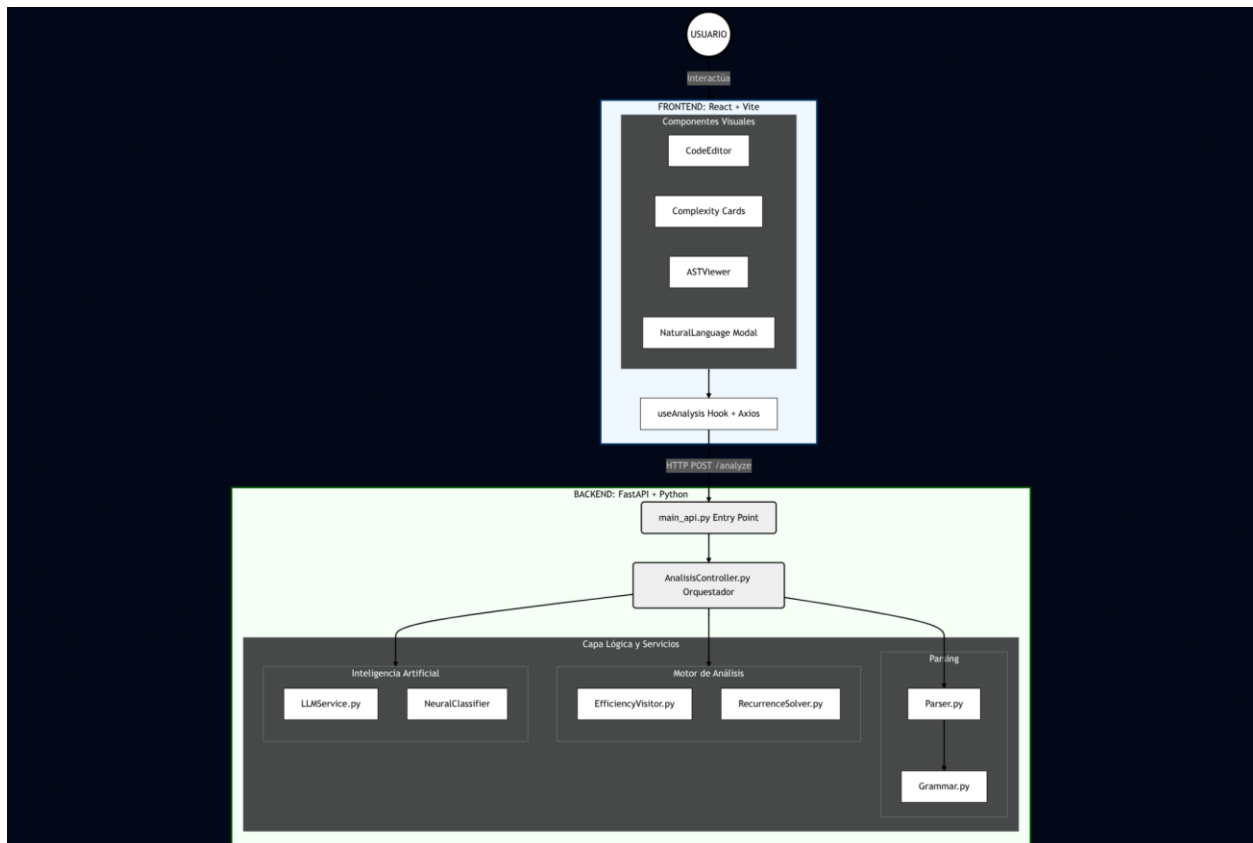
```
# Verificar npm
npm --version
```

Verificar Graphviz
dot -V

3. Arquitectura del Sistema

El sistema opera bajo un patrón Cliente-Servidor (Full-Stack), donde el Backend procesa la lógica pesada y la IA, desacoplado del Frontend.

3.1 Diagrama de Arquitectura de Servicios



3.2 Flujo de Datos para Análisis

1. **Ingreso:** El **Frontend** (React) envía el pseudocódigo a la API REST.
2. **Orquestación:** El **AnalysisController** toma el **{code}** y lo pasa al **Parser**.

3. **Parsing/AST: Parser.py** valida la sintaxis **EBNF** y genera el **Árbol de Sintaxis Abstracta (AST)**.
4. **Detección: Analizador.py** inspecciona el AST para determinar el tipo (Iterativo/Recursoivo).
5. **Análisis Simbólico:**
 - Si es Iterativo: **EfficiencyVisitor** recorre el AST calculando sumatorias SymPy.
 - Si es Recursivo: **RecurrenceSolver** resuelve la ecuación $T(n)$.
6. **Validación Neuronal (Paralelo): NeuralClassifier** extrae *features* del código y clasifica la complejidad predicha.
7. **Validación LLM: LLMService** envía el pseudocódigo y el resultado simbólico a **Google Gemini** (con el PDF de Cormen como contexto) para obtener una explicación y confirmación semántica.
8. **Respuesta:** El Controller consolida los resultados y los envía al Frontend para su visualización.

4. Estructura del Código

4.1 Backend (/Backend)

El Backend sigue una estructura por capas (API, Controladores, Servicios, Modelos) y contiene la lógica de negocio central.

Backend/

```
|— main_api.py          # Entry Point de FastAPI
|— requirements.txt
|— neural_model.json    # Pesos serializados del MLP
|
|— api/                 # Capa REST
|   |— routers/
|   |   |— analysis.py    # Endpoints /analyze, /validate
|   |   |— schemas/
|   |       |— analysis.py # Modelos Pydantic
|   |
|   |— Controladores/
|   |   |— AnalisisController.py # Orquestador del flujo
|   |
|   |— Modelos/          # Entidades del Dominio
|   |   |— Parser.py      # Transpila pseudocódigo -> AST
|   |   |— Complejidad.py
|   |   |— Algoritmo.py
```

```

|
├── Servicios/           # Lógica de Negocio/Matemática
│   ├── Grammar.py       # Reglas EBNF
│   ├── EfficiencyVisitor.py # Calculo de T(n) (Iterativo)
│   ├── RecurrenceSolver.py # 7 Métodos de Recurrencia
│   ├── LLMService.py    # Cliente Google Gemini
│   └── NeuralClassifier/ # MLP con NumPy
│       ├── model.py      # Red Neuronal
│       └── features.py    # Extracción de Features

```

4.2 Frontend (/Frontend)

El Frontend es una aplicación SPA (Single Page Application) basada en React y construida con Vite.

```

Frontend/
├── index.html
├── package.json
├── src/
│   ├── main.tsx          # Punto de entrada de React
│   ├── App.tsx           # Layout principal
│   │
│   ├── components/
│   │   ├── editor/
│   │   │   ├── CodeEditor.tsx # Editor con sintaxis CORMEN
│   │   │   └── NaturalLanguageModal/
│   │   └── analysis/         # Visualización de Resultados
│   │       ├── ComplexityCards/
│   │       ├── ResolutionSteps/
│   │       ├── ASTViewer/
│   │       └── DownloadReport/
│   │
│   ├── hooks/
│   │   └── useAnalysis.ts     # Lógica de estado y llamadas a API
│   │
│   └── services/
│       └── api.ts            # Cliente Axios (interfaz con :8000)

```

5. Dependencias

5.1 Backend (Python)

Paquete	Versión	Propósito
`fastapi`	≥ 0.100	Framework API REST de alto rendimiento
`uvicorn`	≥ 0.23	Servidor ASGI de producción/desarrollo
`lark`	≥ 1.1	Generación de parser (Gramática EBNF)
`sympy`	≥ 1.12	Cálculos simbólicos y simplificación de sumatorias
`google-genai`	$\geq 1.0.0$	SDK oficial para Google Gemini API
`numpy`	≥ 1.24	Base para el MLP de `NeuralClassifier`
`graphviz`, `pydot`	$\geq 0.20, \geq 1.4$	Generación y manipulación de grafos (AST visual)
`python-dotenv`	≥ 1.0	Gestión de la clave GOOGLE_API_KEY

5.2 Frontend (Node.js/TypeScript)

Paquete	Versión	Propósito
`react`	19.2.0	Librería de componentes UI
`axios`	1.13.2	Cliente HTTP para consumir la API de FastAPI
`katex`	0.16.25	Renderizado de fórmulas matemáticas (LaTeX)
`typescript`	~ 5.9.3	Lenguaje base para el Frontend
`vite`	7.2.4	Herramienta de construcción y servidor de desarrollo

5.3 Dependencias del Sistema Operativo

Se requiere la instalación de la herramienta **Graphviz** a nivel de sistema operativo para la generación de las imágenes del AST:

```
# Linux (Debian/Ubuntu)
sudo apt update
sudo apt install graphviz
```

```
# macOS (Homebrew)
brew install graphviz
```

6. Instalación

6.1 Clonar Repositorio


```
# Clonar el proyecto
git clone https://github.com/dsalazardev/EffiCode-Analyzer.git
cd EffiCode-Analyzer
```

6.2 Instalación del Backend (Python)

El Backend debe ejecutarse en un entorno virtual (venv):

```
# Navegar al directorio Backend
cd Backend

# 1. Crear y activar entorno virtual
python3 -m venv venv
source venv/bin/activate # Linux/macOS
# .\venv\Scripts\activate # Windows

# 2. Instalar dependencias
pip install -r requirements.txt

# Verificar instalación (opcional)
python -c "import fastapi, lark, sympy, google.genai; print('✅ Dependencias Python OK')"
```

6.3 Instalación del Frontend (Node.js)

```
# Navegar al directorio Frontend
cd ../Frontend

# 1. Instalar dependencias de Node.js
npm install

# 2. Verificar build (opcional)
npm run build && echo "✅ Build exitoso. Listo para correr."
```

7. Configuración

7.1 Variables de Entorno (.env)

Es obligatorio crear un archivo `.env` en el directorio `/Backend` para gestionar la clave de la

API de Google.

```
cd Backend  
touch .env
```

Contenido requerido para `.env`:

Code snippet

```
# /Backend/.env  
  
# CLAVE DE ACCESO A GOOGLE GEMINI (REQUERIDO)  
GOOGLE_API_KEY=tu_api_key_aqui  
  
# Configuración opcional  
LOG_LEVEL=INFO
```

7.2 Archivo de Contexto (Cormen PDF)

El **LLMService** utiliza el PDF de Cormen como base de conocimiento. Este archivo debe estar en: "Backend/Documentos/Introduction_to_Algorithms_by_Thomas_H_Coremen.pdf"

Nota: El sistema gestiona este archivo de forma inteligente: lo sube a la API de Google Files y lo cachea por 48 horas para evitar el re-procesamiento y el costo de ancho de banda en cada reinicio.

7.3 Configuración de CORS

Para asegurar la comunicación entre **localhost:5173** (Frontend) y **localhost:8000** (Backend), la aplicación utiliza **CORSMiddleware** en `main\_api.py`.

8. Ejecución

8.1 Modo Desarrollo

Terminal 1: Backend (API)

```
cd Backend
source venv/bin/activate
python main_api.py
```

Terminal 2: Frontend (UI)

```
cd Frontend
npm run dev
```

URLs de Acceso:

URL	Propósito
http://localhost:5173	Interfaz de Usuario (Frontend)
http://localhost:8000/docs	Documentación Swagger de la API
http://localhost:8000/analysis/health	Verificar estado de los servicios

8.2 Modo Producción

Se utiliza **uvicorn** con múltiples *workers*:

```
cd Backend

# Ejecutar con 4 workers (se recomienda activar el venv)
uvicorn main_api:app --host 0.0.0.0 --port 8000 --workers 4
```

9. API REST - Endpoints

9.1 Resumen de Endpoints

Método	Endpoint	Descripción
POST	<code>`/analyze`</code>	\textbf{Principal:} Ejecuta el análisis simbólico y neural
POST	<code>`/validate`</code>	Valida un resultado conocido usando Google Gemini
POST	<code>`/analysis/translate`</code>	Traduce lenguaje natural a pseudocódigo
POST	<code>`/analysis/report/pdf`</code>	Genera el reporte final de análisis en PDF

9.2 Análisis de Pseudocódigo (/analyze)

Request Example:

```
{
  "pseudocode": "INSERTION-SORT(A, n)\n  for j ← 2 to n do\n    key ← A[j]\n    i ← j - 1\n  while i > 0 and A[i] > key do\n    A[i + 1] ← A[i]\n    i ← i - 1\n    A[i + 1] ← key\n  return A"
}
```

Response Example (partial):

```
{
  "complexity_o": "O(n²)",
  "complexity_omega": "Ω(n)",
  "complexity_theta": "Θ(n²)",
  "justification_data": {
    "worst_case_function": "T(n) = n(n-1)/2",
    "resolution_steps": [
      {
```

```

    "step": 1,
    "title": "Análisis de Bucles",
    "latex": "T(n) = \sum_{j=2}^n (j-1)",
    "explanation": "...
  }
]
},
"ast_image": "data:image/png;base64,..."
}

```

10. Solución de Problemas

10.1 Errores de Dependencia

Error	Causa	Solución
`ModuleNotFoundError: google.genai`	No se instaló la dependencia del LLM.	`pip install "google-genai>=1.0.0"``
`ImportError: DLL load failed while importing pydot`	$\texttt{\textbf{Graphviz}}$ no está instalado a nivel de SO.	Instalar `graphviz` (ver Sección 5.3).
`ModuleNotFoundError: lark`	Entorno virtual no activado.	`source venv/bin/activate`

10.2 Errores de Configuración

Error	Causa	Solución
`GOOGLE_API_KEY not found`	El archivo `.env` no existe o la clave es incorrecta/está vacía.	Crear/Verificar `.env` con clave válida.

<code>`CORS policy blocked`</code>	El frontend (<code>`:5173`</code>) no está permitido en <code>`allow_origins`</code> del <code>`main_api.py`</code> .	Añadir la URL del frontend a la lista de orígenes.
------------------------------------	---	--

10.3 Instrucciones de Diagnóstico

- 1. `\textbf{Verificar API Key (Directo):}`
Bash
`cd Backend`
`source venv/bin/activate`
`python -c "import os; print(os.getenv('GOOGLE_API_KEY'))"`
- 2. `\textbf{Verificar Estado del Sistema (Health Check):}`
Bash
`curl http://localhost:8000/analysis/health`
Respuesta esperada: `{"status": "healthy", "services": {...}}`

Apéndice: Comandos Rápidos

Operación	Linux / macOS
<code>\textbf{Setup Completo}</code>	<code>`cd Backend && python3 -m venv venv && source venv/bin/activate && pip install -r requirements.txt && cd ../Frontend && npm install`</code>
<code>\textbf{Ejecutar Backend}</code>	<code>`cd Backend && source venv/bin/activate && python main_api.py`</code>
<code>\textbf{Ejecutar Frontend}</code>	<code>`cd Frontend && npm run dev`</code>
<code>\textbf{Generar Build}</code>	<code>`cd Frontend && npm run build`</code>

Información de Contacto

Rol	Nombre
Desarrolladores	● Daner Alejandro Salazar Colorado ● Jaime Andres Cardona Diaz
Repositorio	EffiCode-Analyzer
Curso	Análisis y Diseño de Algoritmos